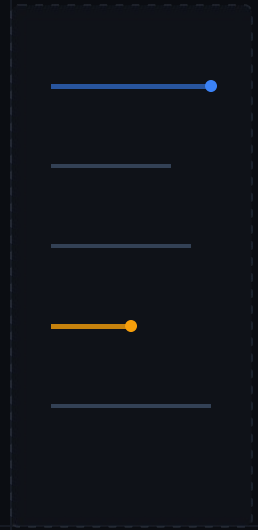
KERNEL RECIPES '26

Kernel Recipes 2026

User Space Assisted Scheduling with Sched QoS

Qais Yousef
Google | qyousef@layalina.io



Scheduler Challenges

01 / HARDWARE & WORKLOAD DIVERSITY

Large Spectrum

Modern workloads run on Server, Desktop, and Mobile systems with fundamentally different performance, power and thermal capabilities. Workloads running on them vary wildly too.

02 / USER EXPECTATIONS

The {HW, Workload, User} Tuple

End users have wildly different expectations for the same workload. Defining global "best" behavior is impossible as requirements vary vastly.

03 / HISTORICAL MISFOCUS

Addressing the Wrong Problem

Focus has historically been on tuning the kernel scheduler heus. However, attempting to fix this purely within kernel space addresses the wrong problem and will always end up with partial solutions.

One universal scheduler is possible with the right userspace collaboration model

*"The scheduler **cannot** fix the problem on its own; it is fundamentally a **userspace management** problem that requires **kernel assistance** to be fully fixed."*

Examples: What Do Users/Workloads Want?

Workload A needs latency, but workload B wants throughput

One of the major complaints is handling of throughput vs latency. The global setup of the scheduler can bias one over the other, but how can we get them **both**?

Hidden browser tabs and background windows are stealing resources

Some users want to limit interference from workloads they consider not important for their use case.

A group of tasks have dependencies

Cache aware scheduling for example wants to annotate that a group of tasks can benefit from cache locality.

A pipeline of tasks (like UI) work together to achieve a group and collective perf and latency matters.

The scheduler have no idea: it just sees a soup of tasks. Assigning roles, grouping and conditioning are all user space management responsibilities. No heuristic based approach will ever figure this out across the spectrum. The scheduler has to be fair by default.

Policy is for userspace, not kernel space

CURRENT MODEL

Heuristic Ping-Ponging

- The current heuristic-based scheduling approach enforces a specific policy designed for "optimal" behavior under very specific circumstances.
- This creates a **ping-pong effect** where a setup considered good for one user turns out to be a bad setup for another user.
- **Hot swapping scheduler is not viable** because systems must handle all random scenarios running in any combination simultaneously.

PROPOSED MODEL

Userspace QoS & Active Opt-In

- Give user space the mechanisms to explicitly annotate its needs, allowing the scheduler to do its best effort to honor those requirements.
- Provides the necessary means to construct **highly flexible policies** tailored for specific {HW, Workload, User} tuple.
- Ultimately we cannot fix everything automagically within the kernel; userspace must **actively opt-in** to manage these policies effectively and reap the benefit, or the downfall.

Limitations: The Scheduler Needs To Evolve Still

> **Wake-up path limited to load:** Cannot select CPU based on latency or QoS. Needs a multi-modal selection criteria that combines EAS, latency, load, etc.

> **Slow pull-based load balancer:** Reacts slowly to load shifts; we need a new push-based load balancer for rapid response and continuously keeping the system balance based on perf, latency and power, etc.

> **Load balance task placement limited to load:** Load balancer placement logic is different to wake up path. It needs to be wired into the multi-modal wake-up path to unify decision-making.

> **Slow DVFS and TICK response:** Takes multiple ticks to adjust frequencies, compounding latency issues when the TICK is 4ms instead of 1ms.

> **Incomplete generalized inheritance:** Proxy Execution is progressing, but futex_pi is not default in user space yet and incurs performance penalties that needs to be addressed.

4msTICK

SLOW FOR MODERN SYSTEMS

Exacerbates slow scheduler reactions over 1ms.

KEY AREAS

- Multi-modal QoS wake-up
- Push-based load balancer
- Unified task placement
- Full QoS inheritance support

Easy Opt-In: Config-Based Hinting - Zero API Approach

API: What's the right one?

Creating new APIs is challenging due to the need for universal consensus and a chicken-and-egg problem: new APIs require polished data, but cannot gather it without deployment. True for kernel or user space new APIs.

2-3 Years Adoption Delay

Deployment Cycle is Long

- The kernel with new APIs has to become common in production.
- Workloads must opt-in and release new binaries to use it
- Distros have to move to the latest workload version utilizing the interface.

schedqos_daemon

```
// Listen & tag tasks via static QoS
on_process_create(pid) →
apply_qos_hint(pid, static_qos_desc)
```

Daemon-Based Hinting Mechanism

> **Immediate Deployment:** Daemon listens for process/task creations and tags them based on static QoS descriptions without needing API updates.

> **Rapid Iteration:** Modifications take effect instantly upon daemon restart, enabling continuous testing before formal API standardization.

> **Long-Term Flexibility:** You can customize and deploy based on your own need and challenges. Users and admins are in full control.

The QoS Classes

Class	Description	**Suggested** sched_attribute mapping
USER_INTERACTIVE	Requires immediate response.	runtime = 8ms policy = normal rampup_multiplier = 4
USER_INITIATED	Tolerates a short delay, but must respond quickly still.	runtime = 10ms policy = normal rampup_mulitplier = 1
UTILITY	Tolerates long delays, but not prolonged ones.	runtime = 16ms policy = batch rampup_multiplier = 0 uclamp_max = 75%
BACKGROUND	Doesn't mind prolonged delays.	runtime = 20ms policy = batch/idle rampup_multiplier = 0 uclamp_max = 50%
DEFAULT	System default behavior	Same as UTILITY

How fast does USER_INTERACTIVE need to be?

- The mapping can be further tweaked by adding another dimension, **PERIOD**
- Can help the SchedQos library/middleware tweak the mapping
  a. The info can be passed to the kernel via **sched_attr::period** for additional in kernel help
- We can easily scale to deal with **240Hz** refresh rate vs **30Hz** ones

240Hz

Class	Suggested sched_attribute mapping
USER_INTERACTIVE	runtime = 2ms policy = normal rampup_multiplier = 8
USER_INITIATED	runtime = 5ms policy = normal rampup_mulitplier = 2
UTILITY	runtime = 8ms policy = batch rampup_multiplier = 0 uclamp_max = 75%
BACKGROUND	runtime = 20ms policy = batch rampup_multiplier = 0 uclamp_max = 50%

30Hz

Class	Suggested sched_attribute mapping
USER_INTERACTIVE	runtime = 12ms policy = normal rampup_multiplier = 1
USER_INITIATED	runtime = 16ms policy = normal rampup_mulitplier = 1
UTILITY	runtime = 20ms policy = batch rampup_multiplier = 0 uclamp_max = 75%
BACKGROUND	runtime = 30ms policy = batch rampup_multiplier = 0 uclamp_max = 50%

Elements of Success: Dynamic QoS Scaling and Flexibility

How can users/admins and workloads dynamically adjust the mapping to address HW limitations or obtain desired behavior? Several layers of controls are required.

CONTROL SCALING FACTOR

Dynamic Scaling

Dynamic Feedback

schedqos daemon can get runtime feedback about desired period (FPS) to adjust mapping accordingly. Requires Window Manager / UI library integration.

Static Feedback

Users & Admins can explicitly adjust configuration files for specific applications to control target period or FPS settings.

Environmental Feedback

Daemon monitors environmental factors like **thermals, battery state**, etc., automatically scaling based on policy rules.

CONTROL QOS DEFINITIONS

Flexible QoS Classes

Custom QoS Classes

Users/Admins can create **new QoS classes** to address specialized workloads that don't fit default definitions or need time to generalize. RT planning can be done in schedqos if we want!

Custom Mapping

Complete freedom to configure **alternative mappings** to suit specific hardware capabilities or custom application requirements.

Robust Baseline classes + Escape Hatch

Provides a **universal baseline classes** that works out of the box for most cases, while offering full override flexibility for specialized edge cases.

Dynamic Role Management: Group Access Control

Dynamically allow or disallow QoS and introduce custom policies based on the specific group a process currently belongs to.

EXAMPLE SCENARIO

USER_INTERACTIVE

BACKGROUND GROUP

- EFFECT: TASK IS COMPLETELY IGNORED BY SYSTEM

Minimized windows and hidden browser tabs can be automatically moved to a background group.

This deliberately disables their USER_INTERACTIVE tasks, conserving significant power and hardware resources.

Group Implementation

Groups can be configured as **purely virtual** constructs within the framework.

Alternatively, they can be directly backed by **real Linux cgroups**.

Real Cgroups Requirement

Utilizing real cgroups support requires adding a new `netlink` subsystem interface.

This actively informs userspace whenever a task changes its underlying group association for it to impose a custom policy.

Cookies For QoS Grouping

USE CASE 01

Cache Locality

- Enables a group of memory dependent tasks to stay on the same cache level.

USE CASE 02

Pipelines

- Groups tasks working together to produce a single output (e.g., UI pipelines).

USE CASE 03

Producer / Consumer

- Prevents starvation when neither side produces or consumes at the "right rate".

COOKIE MANAGEMENT & TAGGING SCOPE

Per-Process Tagging

Purely userspace managed cookies for individual per-process tagging of its own threads.

Multi-Process Tagging

Requires kernel assistance to generate unique global cookies to ensure accurate tracking across processes.

Be careful: Single QoS Manager

Single Global Controller

To maintain system stability and predictability, we must enforce a single QoS manager at the kernel level. We simply cannot have several global QoS managers operating simultaneously across the system without creating inevitable conflicts.

STRICT ABI ENFORCEMENT

Kernel Interface & ABI Protection

When implementing the dependent kernel QoS interface we need to be extremely careful to enforce its usage to avoid repeating TCMalloc incident.

EXCLUSIVE ACCESS CONTROL

Capability Based Access

Only the QoS manager can use the kernel interface for to avoid random workloads using it in ways that make the ABI hard to evolve. This will be enforced by giving the QoS manager a special capability.

Next Steps

We actively need help from the community to progress faster on this initiative. Key immediate priorities:

> WAKE-UP BASED ON LATENCY
Implement a multi-modal wake-up path based on latency rather than relying on load only.

> PUSH LOAD BALANCER
Develop a new push-based load balance mechanism to improve reaction times.

> SCHEDQOS UTILITY
Add robust cookies support and integrate group ACL support directly into the `schedqos` utility.

> FUTEX_PI DEFAULT LOCKING
Connect userspace locks to `futex_pi` to properly support generalized inheritance without performance penalties.

> DEVELOP KERNEL QOS INTERFACE
Develop Kernel QoS interface for the QoS Manager. Discussion is ongoing and there's an LPC talk in October about it.

Questions & Discussion

Q&A

Thank you! Any questions?

> **LPC TALK**
> Check out the LPC talk:
> https://lpc.events/event/19/contributions/2089/

> **SCHEDQOS UTILITY**
> Try the utility, while still WIP but usable:
> https://github.com/qais-yousef/schedqos

> **LKML ANNOUNCEMENT**
> Read the schedqos alpha release announcement on LKML:
> https://lore.kernel.org/lkml/20260415000910.2h5misvwc45bdumu@airbuntu/

> **SCHEDQOS INTERFACE**
> Kernel interface discussion:
> https://lore.kernel.org/lkml/20260504020003.71306-9-qyousef@layalina.io/